01

920B 与 x86 的性能差距

Rust 标准库跨平台性能分析：原因、代码示例与修改方法。

比较背景

分析对象

比较同一版本 Rust 标准库在 920B 与 x86 上完成相同工作的成本，结合生成代码和独立实验分析差距。

ARM	x86
HiSilicon 920B	**对照平台**
2.9 GHz	相同标准库提交

比较口径

$$R = \text{ARM cycles} / \text{x86 cycles}$$

R > 1 表示 ARM 使用更多周期。实际执行时间还受频率影响。

测试版本

标准库 `07e526db9df`

`rustc 1.101.0-dev · LLVM 23.1.4`

`glibc: ARM 2.38 / x86 2.42`

内容目录

17 页内容

01 **源码实现** 02
NEON 路径与平台实现

02 **LLVM 优化** 03–07
Rust、LLVM IR 与编译选项

03 **硬件差异** 08–15
执行能力、数据依赖与访存成本

04 **待分析现象** 16
malloc / memcpy：仅保留差异数据

05 **测量条件** 17–18
代码位置与系统开销

点击目录进入，或按 → 开始演示。

范围 已验证原因与待分析现象分别标注。malloc、memcpy 页保留测量差异，具体原因尚未分析。

缺少 NEON 路径

02

ARM · 当前基线 cycles/次	x86 · 当前基线 cycles/次	R · 已含 NEON 的长串比较
1318.9	1417.6	0.93

原因与示例

RUST

早期源码仅为 x86 SSE2 启用至少 16 B 的大小写无关分块比较，ARM 因平台条件限制进入逐字节路径，每个字节都要判断并更新索引。添加 NEON 路径后，ARM 也能一次处理 16 B。当前测试版本已包含该改动。

示例代码

```
#[cfg(all(target_arch = "x86_64",
          target_feature = "sse2"))]
if a.len() >= 16 {
    return a.eq_ignore_ascii_case_chunks::<16>(b);
}
a.eq_ignore_ascii_case_simple(b)
```

修改方法

已合入

添加 AArch64 NEON 路径。is_ascii 每 64 B 合并四个向量后归约；大小写无关比较复用 16 B 分块辅助函数。

平台条件　64 B 主循环

```
#[cfg(any(
    all(target_arch = "x86_64", target_feature = "sse2"),
    all(target_arch = "aarch64", target_feature = "neon")
))]
```

BENCH ascii::*::case00_libcore · str::eq_ignore_ascii_case::bench_large_str_eq

范围 跨机对照为 str::eq_ignore_ascii_case::bench_large_str_eq，已包含 NEON。原 PR 的 ARM 长串对照为 6235.89→546.61 ns，属于另一组版本对照。

PR #160143 ↗

随机条件被转换为分支

03

ARM · 关闭优化前 → 后	x86 · 默认 cycles/次	R · ARM 修改前 → 后
1830893 → 1160030	1197668	1.53 → 0.97

原因与示例

CODE / LLVM IR

堆调整需要比较左右子节点，再选择较大的一个。LLVM 将这个选择编译为条件跳转；随机输入下，左右选择难以预测，预测失败后需要丢弃错误路径的执行结果。关闭 select-optimize 后使用 cinc 计算索引，branch-miss 从 89849 降至 22696 次。

Rust 代码

```
let use_right = data[child] <= data[child + 1];
child += usize::from(use_right);
let selected = data[child];
```

对应 LLVM IR

```
%l = load i32, ptr %pl
%r = load i32, ptr %pr
%le = icmp ule i32 %l, %r
%z = zext i1 %le to i64
%c = add i64 %child, %z
%pc = getelementptr inbounds i32, ptr %d, i64 %c
%v = load i32, ptr %pc
```

BENCH binary_heap::bench_from_vec · bench_find_smallest_1000

修改方法

已验证

关闭 select-optimize，保留无分支选择。该开关用于单项对照，需分别检查其他负载。

编译选项

```
rustc -O \
  -C llvm-args=-aarch64-select-opt=false \
  bench.rs
```

范围 随机输入中 ARM branch-miss 为 89849→22696，约减少 75%；x86 为 19617。统计单位均为一次完整对照。

可预测分支被转换为条件选择

04

ARM · cycles/字符 · 前 → 后	x86 · 同组俄文对照	R · ARM 修改前 → 后
8.159 → 1.804	**2.345**	**3.48 → 0.77**

原因与示例

CODE / LLVM IR

UTF-8 扫描先读首字节，再确定字符长度并计算下一地址。LLVM 将两字节路径改为 csel 后，形成“读取首字节 → 比较 → 选择长度 → 计算下一地址”的依赖链。俄文输入中字符通常占两字节，保留可预测分支可以提前执行后续加载，缩短等待。

Rust 代码

```rust
let len = if first < 0x80 { 1 }
    else if first < 0xe0 { 2 }
    else if first < 0xf0 { 3 }
    else { 4 };
i += len;
```

对应 LLVM IR

```llvm
%two = icmp ult i8 %b, -32
br i1 %two, label %latch, label %big
; 省略三、四字节路径
latch:
  %len = phi i64 [1, %loop], [2, %multi], [%l34, %big]
  %i2 = add i64 %i, %len
```

BENCH str::char_count::ru_huge::case03_manual_char_len

修改方法

已验证

关闭 early-ifcvt，保留可预测的两字节字符分支，使后续加载有机会提前执行。

编译选项

```
rustc -O \
  -C llvm-args=-aarch64-enable-early-ifcvt=false \
  bench.rs
```

范围 使用 L02 的同组俄文对照：ARM 8.159→1.804，x86 2.345 cycles/字符。IR 节选省略三、四字节路径。

循环结束判断每轮多一条加法

05

ARM · cycles/元素 · 前 → 后	x86 · cycles/元素	R · ARM 修改前 → 后
1.873 → 1.036	**1.006**	**1.86 → 1.03**

原因与示例

CODE / LLVM IR

同一函数有两条循环时，LLVM 的循环强度削减（LSR）将 next == 1000000 改为 next - 998999 == 1001。常量提升将偏移 -998999 作为公共值复用，后续指令选择无法再把它合并回直接比较，ARM 每轮保留一条额外 add。关闭常量提升后，恢复为索引与结束值的寄存器比较。

Rust 代码

```rust
for i in 1001..1_000_000 {
    consume(i);
}
// 同一函数的另一分支中有第二条同形循环
```

对应 LLVM IR

```llvm
a:
  %x = phi i64 [1001, %entry], [%x2, %a]
  %x2 = add i64 %x, 1
  store volatile i64 %x, ptr %p
  %c1 = icmp eq i64 %x2, 1000000
  br i1 %c1, label %exit, label %a
```

BENCH iter::bench_skip_while_sum · bench_skip_while_ref_sum

修改方法

已验证

关闭 constant-hoisting，避免其与 LSR 组合生成冗余 add。

编译选项

```
rustc -O \
  -C llvm-args=-disable-constant-hoisting \
  bench.rs
```

范围 IR 展示改写前的一条循环；完整复现有两条同形循环。单循环、结束值 4095、关闭 LSR 或常量提升的对照均无这条额外 add。

向量加载与重排选择不佳

06

ARM · cycles/B · 前 → 后	x86 · chars count 对照	R · ARM 修改前 → 后
0.1862 → 0.1486	**0.1499**	**1.24 → 0.99**

原因与示例

CODE / LLVM IR

字符计数中，LLVM 选择四路交错加载 ld4，并将部分寄存器临时值写入栈后再读回。字段抽取中，VectorCombine 将 8 个 i32 的加载缩成 7 个，后续无法生成两路交错加载 ld2，改为逐 lane 加载和搬移。两种选择都增加了数据搬移指令；关闭对应优化可减少这些操作。

Rust 代码

```rust
let fields: Vec<_> = pairs.iter()
    .map(|p| p.0)
    .collect();
```

对应 LLVM IR

```llvm
define void @f(ptr %s, ptr %d) {
  %w = load <8 x i32>, ptr %s, align 4
  %v = shufflevector <8 x i32> %w,
      <8 x i32> poison,
      <4 x i32> <i32 0, i32 2, i32 4, i32 6>
  store <4 x i32> %v, ptr %d, align 4
  ret void
}
```

BENCH str::char_count::*::case00_libcore · vec::bench_map_fast/regular

修改方法

已验证

对 chars count 关闭交错访问选择；对 AoS 关闭 VectorCombine，分别验证完整 benchmark。

字符计数　字段抽取

```
rustc -O \
  -C llvm-args=-enable-interleaved-mem-accesses=false \
  bench.rs
```

范围 上方数据为字符计数关闭交错加载的对照；代码和 IR 为字段抽取案例，关闭 VectorCombine 约快 6%，完整测试仍有其他成本。

栈读写范围不匹配，增加写后读等待

07

ARM · cycles/轮 · 前 → 后	x86 · 默认 cycles/轮	R · ARM 修改前 → 后
20.087 → 4.889	5.086	3.95 → 0.96

原因与示例

CODE / LLVM IR

胖指针的数据指针位于 sp+16，虚函数表指针位于 sp+24。LLVM 先用 stp 配对写入整数和数据指针，再用 str 写入虚函数表指针，随后用 ldp 配对读回两个指针，读取范围跨越两次写入。16 B 的 TypeId 返回区又与 1 B 布尔值、8 B 临时指针复用位置，反复覆盖；两项共同增加 ARM 的写后读等待。

Rust 代码

```rust
let mut x = 0isize;
let mut y = &mut x as &mut dyn Any;
black_box(&mut y);
black_box(y.downcast_ref::<isize>() == Some(&0));
```

对应 LLVM IR

```llvm
%b = alloca [1 x i8], align 1
%p = alloca [8 x i8], align 8
%s = alloca [16 x i8], align 16
; 生命周期依次结束，允许栈槽复用
call void %fn(ptr sret([16 x i8]) %s, ptr %arg)
%v = load i128, ptr %s, align 16
```

BENCH any::bench_downcast_ref

修改方法

已验证

分别关闭栈槽复用或 AArch64 加载/写入配对进行对照。两种全局开关都可能使其他写法回退。

栈槽复用　加载/写入配对

```
rustc -O -C llvm-args=-no-stack-coloring probe.rs
```

范围 固定地址对照：默认 20.13，只调整指针写入配对 13.06，组合调整栈位置并消除重复覆盖 4.99 cycles/轮。具体硬件转发或合并环节尚未完全确定；上方 4.889 来自关闭栈槽复用。

整数指令吞吐较低

08

ARM · 16 条 add 的 cycles/轮	x86 · 相同独立 add 数量	R
4.36	**3.10**	**1.41**

原因与示例

RUST

独立 add 对照中，受测 ARM 每周期约完成 4 条，x86 接近 6 条。迭代器除计算结果外，还要更新索引、剩余数量和游标；这些整数操作越多，越容易受 ARM 较低的吞吐限制。BTreeMap 在叶节点内直接推进索引，可以减少逐元素导航和句柄搬移。

示例代码

```
sum += x;
index += 1;
remaining -= 1;
// 每元素控制操作增加整数执行压力
```

BENCH iter::bench_enumerate_ref_sum · bench_next_chunk_filter_map_even

修改方法

已合入

BTreeMap 在叶节点内直接推进索引；释放时按节点后序处理，减少逐元素导航、句柄搬移和判断。

叶内迭代　按节点释放

```
if index < leaf.len:
    result = leaf[index]
    index += 1
else:
    navigate_to_next_node()
```

范围 修改示例对应 BTreeMap；伪代码省略异常清理。独立 add 结果不能直接推断物理端口数。

PR #163521 ↗　PR #162931 ↗

逐元素筛选与压缩开销

09

ARM · 官方 retain cycles/次	x86 · 官方 retain cycles/次	R · 整体计时含输入重建
231948.8	110900.6	2.09

原因与示例

RUST

首个删除元素出现后，标量 retain 对每个元素判断是否保留；保留时搬移元素并增加写索引。写入位置依赖此前保留的数量，索引更新无法完全独立执行。SVE COMPACT 可按块压缩保留元素，减少搬移阶段的逐元素判断、写入和索引更新。

示例代码

```rust
let mut write = 0;
for read in 0..len {
    if predicate(&values[read]) {
        values[write] = values[read];
        write += 1;
    }
}
```

修改方法

实验路径

Vec::retain 可分块按序计算谓词并保存掩码，再以 SVE COMPACT 批量压缩保留元素，减少搬移阶段的分支和索引更新。

处理流程　SVE u32

```
输入  [10,11,12,13,14,15,16,17]
掩码  [ 1, 0, 1, 0, 1, 0, 1, 0]
输出  [10,12,14,16]
每块：按序计算谓词 → 保存 mask
      → compact → write += kept
```

BENCH vec::bench_retain_100000 · bench_retain_whole_100000

范围 官方输入为交替保留，整体计时包含输入重建；R=2.09 尚未分解到具体操作，不能归因于随机分支误预测。SVE 同条件跨机收益尚未测量；示例省略析构和 panic 清理。

指令依赖链延迟较高

10

ARM · u16 fold 前 → 后	x86 · 相同交错选项前 → 后	R · cycles/元素
0.254 → 0.182	**0.198 → 0.200**	**1.28 → 0.91**

原因与示例

CODE / LLVM IR

extract_bits 每一阶段都使用上一阶段的 x，移位结果还要参与后续异或。受测 NEON 移位、异或的依赖延迟约为 2 cycles，x86 SSE 约为 1，连续多阶段会累积等待。改用等价加法可生成 usra，将右移与累加合并；增加独立累加器也可重叠执行不同数据的运算。

Rust 代码

```
let q = x & masks[stage];
x ^= q;
x ^= q >> n;
```

对应 LLVM IR

```
%notmask = xor <2 x i64> %mask, <i64 -1, i64 -1>
%keep = and <2 x i64> %x, %notmask
%q = and <2 x i64> %x, %mask
%shift = lshr <2 x i64> %q, <i64 1, i64 1>
%r = add <2 x i64> %keep, %shift
```

BENCH num::int_bits::*::extract_bits · fold_u16

修改方法

已合入 / 编译选项

extract_bits 对至多 64 位的 NEON 路径改用等价加法，生成 usra；Latin-1 直接映射减少查表依赖；u16 fold 可增加独立交错工作。

位运算　Latin-1　u16 交错

```
let q = x & masks[stage];
x = (x & !masks[stage])
    .wrapping_add(q >> n);
```

范围 跨机数值为 L14 u16 fold 的 IC=2→4，单位 cycles/元素。extract_bits 的加法示例依赖不重叠位区段条件；u128 保留原式。

PR #164611 ↗　PR #162750 ↗

写后读依赖延迟较高

11

ARM · 依赖链 cycles/轮	x86 · 同地址状态读改写	R
7.07	1.00	7.07

原因与示例

ASM

循环每轮从同一地址读取状态、加一并写回，下一轮读取必须取得前一轮写入的新值，形成跨轮依赖。受测同地址读改写链为 ARM 7.07、x86 1.00 cycles/轮。Vec 的 len 或 RandomState 计数器反复经过这条依赖链时，会限制循环速度。

示例代码

```
ldr x0, [x1]
add x0, x0, #1
str x0, [x1]
// 下一轮再次从同一地址读取
```

BENCH slice::push · string::bench_push_char_two_bytes · hash::map::new_drop

修改方法

原因已复现

无已验证的通用修改方法。局部 len 对照说明减少状态往返有效，但具体接口仍须保持 panic、别名和可观察状态语义。

无已验证的对应修改方法

当前证据用于解释成本或界定测量范围。原因对照和局部原型的结果，分别保留其适用条件。

范围 HashMap 创建测试主要耗在 RandomState 线程局部计数器，R≈6.93。主循环的缺页影响需另行剥离。

x86 专有指令

ARM · 密集命中 cycles/次	x86 · 同宽密集命中对照	R
27.0	16.6	1.63

原因与示例

CODE / LLVM IR

x86 的 pmovmskb 能用一条指令提取 16 个字节各自的最高位，组成整数位掩码，再据此定位首个匹配。受测 ARM NEON 路径没有对应单指令，需要多条向量合并和向通用寄存器传送的指令。命中越密集，反复提取位置的成本占比越高；长无匹配输入中 ARM 可以更快。

Rust 代码

```rust
let v = Simd::<i8, 16>::from_array(bytes);
let mask = v.simd_lt(Simd::splat(0));
let bits = mask.to_bitmask();
```

对应 LLVM IR

```llvm
define i16 @mask(<16 x i8> %v) {
  %c = icmp slt <16 x i8> %v, zeroinitializer
  %m = bitcast <16 x i1> %c to i16
  ret i16 %m
}
```

BENCH str::find_zzz_char · vec::bench_dedup_none_*

修改方法

已验证

无对应已验证修改方法。具体接口是否只需要“有无命中”，会改变完整索引提取的必要性。

无已验证的对应修改方法

当前证据用于解释成本或界定测量范围。原因对照和局部原型的结果，分别保留其适用条件。

范围 Rust 使用 portable SIMD 表示等价操作，需导入 SimdPartialOrd，不代表当前 std 源码。IR 为 L11；长无匹配对照为 ARM/x86 218/277 cycles。

未对齐与跨缓存行访问开销

13

ARM · 未对齐加载额外 cycles	x86 · 相同 16 B 加载额外 cycles	ARM 额外增加的周期
0.30	**0.04**	**+0.26**

原因与示例

TEXT

16 B 加载若从缓存行内偏移 52 B 开始，会读取第 52–67 字节，跨越 64 B 缓存行边界，需要处理两条缓存行的数据。未对齐加载对照中，ARM 每条额外约 0.30 cycle，x86 约 0.04。字段地址和元素步长会决定这类访问出现的频率。

示例代码

```
64 B cache line
|--------------------------------|
                         [52 ... 67]
                         ← 16 B →
                                  ↑ 跨行
```

BENCH slice::random_inserts · vec::bench_map_fast/regular

修改方法

已验证

无对应已验证的通用修改方法。字段加载实验确定了地址偏移的影响，通用改法必须保持所取字段的值。

—

无已验证的对应修改方法

当前证据用于解释成本或界定测量范围。原因对照和局部原型的结果，分别保留其适用条件。

范围 上方为未对齐加载对照。跨缓存行字段实验中，x86 消除跨行访问后为 7993.8→6021.0 cycles，指令数相同；实验字段值相同，地址减 4 不能直接用于任意结构体。

宽整数运算的指令支持不同

14

ARM · 宽除法 cycles/链	x86 · 同一 65÷33 位除法	R
37.19	**19.05**	**1.95**

原因与示例

RUST

示例的分子需要 65 位、分母需要 33 位，超出单个 u64 分子的范围。受测 x86 路径可用硬件除法处理，ARM 则进入 u128 软件除法，执行多条运算和判断。同一除法依赖链为 ARM 37.19、x86 19.05 cycles；该结论只适用于这组值域和生成代码。

示例代码

```rust
let numerator: u128 = (1u128 << 64) + 1;
let denominator: u128 = (1u128 << 32) + 1;
let quotient = numerator / denominator;
```

BENCH u128_sqrt_uniform · num::bench_u64_from_str_radix_10

修改方法

已验证

无对应已验证修改方法。可否缩窄运算必须由算法值域、取整和溢出条件决定；上述 65 位分子不能直接转换为 u64。

无已验证的对应修改方法

当前证据用于解释成本或界定测量范围。原因对照和局部原型的结果，分别保留其适用条件。

范围 常数除法通常已转换为乘法；这组软件除法结论不适用于所有整数格式化。

加载吞吐较低

15

ARM · 独立加载条数/cycle	x86 · 独立加载条数/cycle	ARM 加载吞吐 / x86
≈ 2	≈ 3	≈ 2/3

原因与示例

TEXT

独立加载对照中，受测 ARM 每周期约完成 2 条 64/128 位加载，x86 约 3 条。完成同样数量的加载，ARM 需要更多周期。连续扫描、逐元素读取等加载密集循环更容易受这一吞吐上限限制；算法应减少重复读取和大量小块搬移。

示例代码

```
a = load(input_a[i]);
b = load(input_b[i]);
c = load(input_c[i]);
// 相互独立，用于观察加载吞吐
```

BENCH vec_deque::bench_try_fold · slice::rotate_huge_by9199_big

修改方法

已合入

切片 rotate 对满足条件的大元素按机器字组织连续块交换；剩余一侧可放入栈缓冲时，切换为带缓冲的 memmove。

连续块交换　小尾部处理

```
if 元素足够大且大小、对齐满足条件:
    按多个 MaybeUninit<usize> 组织
    使用连续块交换
else:
    保留原分派
```

范围 独立加载吞吐来自 S4 对照；完整 benchmark 的差距还需结合指令数和访问方式分析。rotate 原 PR 两组大元素测试耗时降低约 48.4%、23.2%；测试基线尚未包含该 PR。

PR #162754 ↗

malloc 与 memcpy 存在跨机差异

16

ARM · 800 B malloc+free cycles/次	x86 · 800 B malloc+free cycles/次	8 KB memcpy · ARM / x86
31.36	17.72	333 / 138–222

原因与示例

C

已测得 malloc 和 memcpy 在 ARM 与 x86 上的耗时不同。本页保留对照数据，具体原因尚未分析。

示例代码

```c
p = malloc(800);
free(p);
q = malloc(8000);
free(q);
// memcpy 的两个缓冲区预先分配
memcpy(dst, src, 8000);
```

BENCH vec::bench_from_slice_0100 · vec_deque::bench_from_array_1000 · vec::bench_extend_*

修改方法

有差异 · 原因未分析

具体原因尚未分析，暂无对应修改方法。

已测得的差异

测试	ARM	x86
800 B malloc+free	31.36	17.72
8000 B malloc+free	70.11	70.40
8000 B memcpy	约 333	约 138 或 220–222

单位：cycles/次。这里只列出测量结果，暂不作具体成因归因。

范围 单位 cycles/次。glibc 为 ARM 2.38、x86 2.42。8 KB memcpy 的 x86 结果随源和目标的相对位置变化；表中保留测量范围。

代码地址改变短循环吞吐

17

ARM · 固定循环 cycles/元素	x86 · 对应地址扫描对照	仅改变代码偏移
1.007–1.627	1.001–1.017	0–60 B

原因与示例

TEXT

保持循环指令、输入和执行次数不变，仅将代码起点移动 0–60 B，ARM 就从 1.007 变为最高 1.627 cycles/元素；对应 x86 对照为 1.001–1.017。实验确认代码位置会改变这组短循环的吞吐，具体取指或预测环节尚未确定。

示例代码

```
.text + 0 B   : same_loop
.text + 16 B  : same_loop
.text + 32 B  : same_loop
.text + 48 B  : same_loop
```

BENCH iter::bench_enumerate_ref_sum · ascii::bench_ascii_escape_display_*

修改方法

测量范围

无通用修改方法。使用多个代码地址复测，分别记录布局与指令变化的贡献。

无已验证的对应修改方法

当前证据用于解释成本或界定测量范围。原因对照和局部原型的结果，分别保留其适用条件。

范围 数值来自 F2 的 a 型循环。各平台固定自己的汇编，只移动起点；b 型 x86 循环在个别位置也会明显变慢。

缺页和调度主导部分计时

18

ARM · spin → yield cycles/来回	x86 · spin → yield cycles/来回	R · spin → yield
11585540 → 11864	**11584 → 8851**	**1000.1 → 1.34**

原因与示例

TEXT

首次写入尚未分配物理页的内存可能触发缺页处理，耗时包含内核分配和页面初始化。同核 pipe 两端若用非阻塞自旋等待，等待线程会占用 CPU，使另一端无法及时运行；改用 yield 后，受测 ARM 往返周期从约 1159 万降至 1.19 万，表明原计时主要受调度等待影响。

示例代码

```
while would_block() {
    spin();
}
// 同核另一线程可能无法及时运行
```

BENCH slice::push · kernel_copy::bench_socket_pipe_socket_copy

修改方法

测量范围

无 LLVM 修法。控制页面触碰与线程调度，计入内核周期；让出 CPU 的结果作为调度对照。

—

无已验证的对应修改方法

当前证据用于解释成本或界定测量范围。原因对照和局部原型的结果，分别保留其适用条件。

范围 同核 pipe 往返的 spin→yield 对照，计入内核周期。系统与调度条件不同；这里比较计时条件的影响，不能据此推断 CPU 执行能力。